Thyroid Cancer Detection with Grad-CAM: Project Documentation

1. Overview and Purpose

This repository is an inference-and-presentation application for classifying thyroid medical images (the README says "ultrasound or pathology") as **Benign** (class 0) or **Malignant** (class 1) with a deep convolutional network, and for explaining each prediction with a Grad-CAM attention heatmap. It is not a training project. The trained network is not stored in the repository; it is downloaded at start-up from the HuggingFace Hub (`vaidehibh/fibonacci_cnn`, file `thyroid_cancer_model.h5`).

The repository contains three user-facing layers built around that one model:

- A **FastAPI REST backend** (`app.py`, `api/`) exposing health, model-info, predict, Grad-CAM, combined analyze, and PDF/JSON report endpoints. Images travel as base64 strings inside JSON.
- A **React + Vite + Tailwind single-page frontend** (`frontend/`) that uploads an image, calls `/api/v1/analyze`, and renders metrics, risk assessment, Grad-CAM panels, and report downloads.
- A **legacy Streamlit app** (`main_streamlit.py`) that does the same job in one process. The README says it was the original version and that the project was "migrated" to FastAPI; a public Streamlit Cloud URL is mentioned in the README.

Besides classification, the system maps the raw sigmoid score to a four-level "risk assessment" with a canned clinical recommendation (for example "Immediate specialist consultation and biopsy recommended"). Every artefact carries a disclaimer that it is for research and education and is not FDA approved.

Important scope notes, based only on what is in the repo:

- There are **no Jupyter notebooks**, no training scripts, no dataset, and no evaluation outputs. The `.gitignore` explicitly ignores `*.ipynb`, `*.csv` and `*.xlsx`, so any such material was deliberately excluded from version control.
- No accuracy, AUC, sensitivity or specificity figure is documented anywhere. The README's "Performance Metrics" section lists only input resolution, inference time (~1-3 s) and memory (~500 MB).
- Git history in the clone has a single visible commit (`e9cd372`, 2026-02-15, "Add Streamlit deployment section").

2. Tech Stack

Area	Technology	Where used
Language	Python 3.8+ (per README)	backend, Streamlit
API framework	FastAPI >= 0.109, Uvicorn[standard] >= 0.27	`app.py`, `api/`
Validation	Pydantic >= 2.5, python-multipart	`api/schemas.py`
ML framework	TensorFlow >= 2.12 with Keras 3 API (`keras.saving`)	`model/`, `utils/gradcam.py`
Model hosting	`huggingface-hub` >= 0.20	`model/model_loader.py`
Model file format	HDF5 (`.h5`), h5py >= 3.8	downloaded model
Imaging	Pillow >= 10, opencv-python-headless >= 4.8, matplotlib >= 3.7 (Agg backend)	preprocessing, overlays
Reports	reportlab >= 4.0	`utils/pdf_generator.py`
Legacy UI	Streamlit >= 1.28	`main_streamlit.py`
Frontend	React 18, Vite 5, Tailwind CSS 3.4, axios, react-dropzone, react-hot-toast, Headless UI, Heroicons	`frontend/`
Frontend tooling	ESLint 8, PostCSS, Autoprefixer	`frontend/`
System packages	`libgl1-mesa-glx`, `libglib2.0-0`	`packages.txt` (Streamlit Cloud)
Testing	`requests`-based smoke script	`test_api.py`

`requirements.txt` uses only lower bounds (`>=`) and has no lock file for the Python side. `requests` (needed by `test_api.py`) is not listed in it. The frontend has a `package-lock.json` (about 206 KB).

3. Repository Structure

```
Thyroid-Cancer-Detection-with-GradCam/
|-- app.py                      FastAPI entry point, CORS, startup model load
|-- config.py                   HF repo id, filename, input size, upload limits
|-- requirements.txt            Python dependencies (lower bounds only)
|-- packages.txt                apt packages for Streamlit Cloud
|-- main_streamlit.py           Legacy single-process Streamlit UI
|-- test_api.py                 Manual smoke tests against a running API
|-- README.md                   Main documentation (API-centric)
|-- QUICKSTART.md               Migration notes and quick start
|-- SETUP_GUIDE.md              Longer setup guide
|-- .gitignore
|-- api/
|   |-- __init__.py            Re-exports router
|   |-- schemas.py             Pydantic request/response models
|   |-- routes.py.old          Superseded monolithic routes (dead file)
|   `-- routes/
|       |-- __init__.py        Exports router, set_model
|       |-- routes_endpoints.py   Endpoint handlers
|       `-- routes_functions.py   Helpers: validate, classify, risk, Grad-CAM viz
|-- model/
|   |-- __init__.py            Imports from model_loader and custom_layers
|   |-- model_loader.py        Custom layers actually used + HF download + load
|   `-- custom_layers.py       Alternative layer definitions (not used at load)
|-- utils/
|   |-- __init__.py
|   |-- gradcam.py             make_gradcam_heatmap
|   |-- image_utils.py         base64 decode/encode, preprocessing for API
|   |-- processing.py          preprocessing for Streamlit upload objects
|   |-- pdf_generator.py       ReportLab PDF report
|   `-- logger_config.py       File + console logging
|-- test files/
|   |-- 0000.jpg               sample image
|   `-- 0378.jpg               sample image
`-- frontend/                  React SPA (details below)
```

Frontend layout:

```
frontend/
|-- package.json, vite.config.js, tailwind.config.js, postcss.config.js
|-- eslint.config.js, jsconfig.json, index.html, README.md
`-- src/
    |-- main.jsx, App.jsx
    |-- context/    PredictionContext.jsx, ThemeContext.jsx
    |-- hooks/      useAnalysis.js
    |-- services/   api.js
    |-- utils/      helpers.js
    |-- styles/     index.css
    `-- components/
        |-- Layout/         Container, Header, Footer
        |-- Upload/         ImageUpload, ImagePreview
        |-- Results/        PredictionMetrics, ClinicalAssessment, MedicalDisclaimer
        |-- Visualization/  GradCAMPanel, InterpretationGuide
        |-- Reports/        DownloadButtons
        `-- UI/             Alert, Button, Card, LoadingSpinner
```

Binary and large files (not read in detail)

File	Size
`test files/0000.jpg`	45,450 bytes (about 44 KB)
`test files/0378.jpg`	49,490 bytes (about 48 KB)
`frontend/package-lock.json`	206,468 bytes (about 202 KB)
`thyroid_cancer_model.h5` (HuggingFace, not in repo)	size not available offline

No model weights are committed. At runtime the file is cached under `./model_cache` (git-ignored).

4. Architecture and ML Pipeline

```
Browser (React SPA, :3000)                    Streamlit UI (legacy, separate process)
  |  file -> base64 (FileReader)                 |  st.file_uploader -> PIL
```

```
  |  POST /api/v1/analyze  (JSON)                |
  v                                              v
+------------------------------ FastAPI (:8000) ------------+   utils/processing.py
| CORS(*) -> router /api/v1 -> validate(len<=10,485,760)     |   (same math, file object)
|   preprocess_base64_image:                                |
|     b64 decode -> PIL -> RGB -> resize 224x224 LANCZOS    |
|     -> float32 / 255.0 -> shape (1,224,224,3)             |
|   model.predict -> p = sigmoid score in [0,1]             |
|   classify:  class = (p >= 0.5)                           |
|   assess_risk: p thresholds 0.25 / 0.75 -> text           |
|   get_conv_layer: last layer whose name contains 'conv2d' |
|   make_gradcam_heatmap(img, model, layer)                 |
|   create_gradcam_vis: jet colormap, 0.6/0.4 blend -> PNGs |
+-----------------------------------------------------------+
        ^                                 |
        | start-up only                   v
 HuggingFace Hub                JSON {prediction, risk, recommendation,
 vaidehibh/fibonacci_cnn          gradcam:{layer_used, images{original,
 thyroid_cancer_model.h5          heatmap, overlay} base64 PNG}}
 (cache ./model_cache)           or PDF (ReportLab) / JSON file download
```

Control flow at start-up: `app.py` calls `load_model()` inside a FastAPI `startup` event, injects the model into the routes module via `set_model`, then runs a smoke prediction on `np.random.rand(1,224,224,3)`. Any failure re-raises and prevents the server from starting.

Control flow per request: the handler validates input, preprocesses, runs a single forward pass, derives class and risk text, and (for Grad-CAM) runs a second forward pass inside a `GradientTape` through a sub-model that exposes the chosen layer's output.

5. Detailed Module Walkthrough

5.1 `config.py`

Plain constants: `HF_REPO_ID="vaidehibh/fibonacci_cnn"`, `HF_MODEL_FILENAME="thyroid_cancer_model.h5"`, `MODEL_INPUT_SIZE=(224,224)`, `MODEL_INPUT_CHANNELS=3`, `MAX_UPLOAD_SIZE_MB=10`, `ALLOWED_EXTENSIONS=['png','jpg','jpeg']`, `LOG_LEVEL`, `LOG_FORMAT`. Only the two HF constants are actually imported (by `model_loader.py`); the others are unused, and the code hard-codes 224 and 10 MB in several places.

5.2 `model/model_loader.py`

Defines three Keras layers registered with `@keras.saving.register_keras_serializable()` so the saved model can be deserialised, then `load_model()`.

- `SEBlock(ratio=16)`: Squeeze-and-Excitation. In `build` it creates GlobalAveragePooling2D, `Dense(channels // ratio, swish)`, `Dense(channels, sigmoid)` and a `Reshape((1,1,channels))`. `call` multiplies the input by the learned channel weights.
- `Avg2MaxPooling(pool_size=3, strides=2, padding='same')`: runs AveragePooling2D and MaxPooling2D, then returns `BatchNorm(avg - 2*max)`. This is the actual "Avg2Max" definition used for loading.
- `DepthwiseSeparableConv(filters, kernel_size=3, strides=1, se_ratio=16, reg=0.001)`: DepthwiseConv2D (same padding, L2) -> 1x1 Conv2D (L2) -> BatchNorm -> swish -> `SEBlock`. If `strides != 1` a 1x1 projection conv is created for the residual path. The output is `x + residual` only when shapes match, otherwise `x`.
- `load_model()`: `hf_hub_download(repo_id, filename, cache_dir="./model_cache")`; checks existence and read access; `keras.saving.load_model(path, custom_objects={...}, compile=False)`; verifies it has `predict`; recompiles with `optimizer='adam'`, `loss='binary_crossentropy'`, `metrics=['accuracy']`; logs layer and parameter counts. Known I/O errors are re-raised; anything else becomes `RuntimeError`.

The recompile is only a convenience: the original training optimiser, learning rate and schedule are not recoverable from this code.

5.3 `model/custom_layers.py`

Contains `CustomLSTM` (subclass of Keras LSTM that drops `time_major`; the comment says it is "same fix as heart murmur project"), a different `Avg2MaxPooling` (average of avg and max pooling, `(avg+max)/2`), and a different `DepthwiseSeparableConv` (no BatchNorm, no SE, no residual, no strides projection). These definitions disagree with the ones in `model_loader.py`, are not passed to `load_model`, and appear to be leftovers from another project. `model/__init__.py` imports both modules, so two classes named `Avg2MaxPooling` and `DepthwiseSeparableConv` coexist. The README architecture diagram mentions "Custom LSTM Layers", which no code path uses.

5.4 `utils/image_utils.py`

- `decode_base64_image(str)`: strips a `data:...,` prefix if a comma is present, base64-decodes, opens with PIL.
- `encode_image_to_base64(PIL, format="PNG")` and `encode_numpy_to_base64(array)`: the latter scales to uint8 (multiply by 255 if max <= 1.0, else cast) and PNG-encodes.
- `preprocess_base64_image(str, target_size=(224,224))`: converts to RGB, LANCZOS resize, `img_to_array/255.0`, adds the batch axis. Returns the tuple `(array, resized PIL image)`. Its type hint says `np.ndarray`, which is inaccurate.

5.5 `utils/processing.py`

`preprocess_image(uploaded_file, target_size)` for the Streamlit path. Same pipeline (RGB, 224x224 LANCZOS, divide by 255, batch axis). No ImageNet-style mean/std normalisation anywhere; whether the model was trained with `/255` scaling is not stated in the repo.

5.6 `utils/gradcam.py`: `make_gradcam_heatmap`

Signature: `make_gradcam_heatmap(img_array, model, last_conv_layer_name, pred_index=None)`. Steps as implemented:

1. Validates that inputs are non-empty and the array is 4-D; looks up the layer via `model.get_layer`.
2. Builds `grad_model = Model(model.inputs, [layer.output, model.output])`.
3. Under `GradientTape`, calls `grad_model(img_array)` and takes `class_output = predictions[:, 0]` (the single sigmoid unit).
4. `grads = tape.gradient(class_output, conv_outputs)`; global-average-pools the gradients over batch and spatial axes to get channel weights.
5. Multiplies each channel of the feature map by its weight (explicit Python loop), averages over channels, applies ReLU (`np.maximum(..., 0)`), and normalises by the maximum if it is positive.
6. Raises on NaN/Inf; logs a warning if all zeros.

Observations: `pred_index` is computed (0 or 1) and logged but never used. The gradient is always that of the malignant-probability output, so for a Benign prediction the map shows regions that push the score toward malignancy, not regions supporting "benign". Also `conv_outputs` is not watched explicitly; that works because it is an intermediate output of the same forward pass.

5.7 `api/schemas.py`

Pydantic models: `ImageRequest(image, filename)`, `GradCAMRequest(+layer_name)`, `AnalyzeRequest(+include_gradcam=True)`, and responses `PredictionResponse`, `GradCAMResponse`, `AnalyzeResponse`, `HealthResponse`, `ModelInfoResponse`, `ErrorResponse` (the last is not used). `prediction` and `images` are untyped `dict`s.

5.8 `api/routes/routes_functions.py`

- `set_model` / `get_model`: module-global model with a fallback `__import__('app').get_model()`.
- `validate(img)`: 503 if no model; 400 if the string is empty or longer than 10,485,760 characters (this limits the base64 text length, so the decoded image limit is about 7.5 MB).
- `classify(p)`: `class = int(p >= 0.5)`, label text, confidence percentage `p` or `1-p` times 100.

- `assess_risk(class, p)`: malignant: `p >= 0.75` High Risk ("Immediate specialist consultation and biopsy recommended") else Moderate Risk ("Further diagnostic tests and specialist review advised"). Benign: `p <= 0.25` Low Risk ("Routine monitoring recommended") else Borderline ("Follow-up imaging in 6-12 months advised").
- `get_conv_layer(model, name=None)`: returns the user-supplied name unchecked, otherwise the last layer whose lowercase name contains `conv2d`; 400 if none.
- `create_gradcam_vis(heatmap, original_pil)`: resizes both to 224x224, applies matplotlib `jet`, blends `0.6*image + 0.4*heatmap` with OpenCV, and returns base64 PNGs `original`, `heatmap`, `overlay`.
- `predict_image(img, filename)`: preprocess, `model.predict`, rejects NaN/Inf, logs filename and score.
- `get_timestamp()`: local time string.

5.9 `api/routes/routes_endpoints.py`

Endpoint	Method	Behaviour
`/api/v1/health`	GET	status, `model_loaded`, timestamp, version "2.0.0"
`/api/v1/model-info`	GET	input/output shape, `count_params()`, class map, conv-layer names
`/api/v1/predict`	POST	class, label, score, confidence %, risk, recommendation
`/api/v1/gradcam`	POST	optional `layer_name`; returns layer used, three PNGs, prediction
`/api/v1/analyze`	POST	predict plus optional Grad-CAM (failure is logged and skipped)
`/api/v1/report/pdf`	POST	runs `analyze_complete`, builds PDF, streams as attachment
`/api/v1/report/json`	POST	runs `analyze_complete`, streams JSON attachment

The README endpoint table omits the two `/report/*` routes even though the frontend uses them. `model-info` uses a bare `except:` that converts all errors into 503. `/report/pdf` and `/report/json` wrap all exceptions, including `HTTPException`, into a generic 500.

5.10 `utils/pdf_generator.py`

`generate_pdf_report(prediction_data, gradcam_data=None) -> BytesIO`. Letter-size ReportLab document: title, metadata table (time, filename), prediction table (green or red header cell by class), colour-coded risk table, optionally a page with the Grad-CAM overlay (4x4 inch) and explanation, a boxed medical disclaimer ("NOT FDA-approved", liability disclaimer) and a footer with "v2.0.0". The user-supplied filename is inserted into the table as plain text.

5.11 `utils/logger_config.py`

`configure_logging()` creates `logs/`, then `logging.basicConfig(level=INFO)` with a `logs/app.log` file handler and a stream handler. Logs include filenames, image size/mode and prediction scores.

5.12 `main_streamlit.py`

Loads the model with `@st.cache_resource`, accepts png/jpg/jpeg, and on "Analyze Image" runs preprocessing, prediction, a results panel, a 3-panel matplotlib Grad-CAM figure (original, heatmap, overlay with alpha 0.4), and a JSON download button. It selects the layer the same way (last name containing `conv2d`). Notable defects: the risk-message lookup iterates a dict keyed `(class, threshold)`; for class 0 the first key `(0, .25)` tests `pred <= 0.75`, which is always true for benign predictions, so the "Borderline" branch is unreachable and every benign result shows "LOW RISK". The downloaded JSON report uses a separate, correct threshold list, so the on-screen message and the report can disagree. The Streamlit message also says "Moderate Risk" for any malignant score under 0.75.

5.13 `test_api.py`

A script (not a pytest suite, no assertions beyond status codes) that calls health, model-info, then predict, gradcam (saving PNGs to `test_outputs/`) and analyze. It looks for `test_image.jpg`, `test_image.png`, `thyroid_scan.jpg`, `sample.jpg` in the working directory; the two bundled samples in `test files/` are not among those names, so the

image tests are skipped by default.

5.14 Frontend

- `services/api.js`: axios client with a hard-coded `baseURL` of `http://localhost:8000`, 60 s timeout; `fileToBase64` (FileReader, strips the data-URL prefix); wrappers for health, model-info, predict, gradcam, analyze and both report downloads (`responseType: 'blob'`).
- `hooks/useAnalysis.js`: `performAnalysis` calls `analyzeImage(file, true)`, stores the result and Grad-CAM images in context, and shows toasts; error text comes from `err.response.data.detail`.
- `context/PredictionContext.jsx` and `ThemeContext.jsx`: global state (uploaded image, result, Grad-CAM, loading, error) and dark-mode theme.
- `utils/helpers.js`: allowed MIME types, 10 MB limit, `validateFile`, size formatting, `getRiskColor` (red >= 0.75 malignant, orange malignant, green benign <= 0.25, blue otherwise), `downloadBlob`, timestamped filenames.
- Components: `ImageUpload` (react-dropzone, 10 MB cap), `ImagePreview`, `PredictionMetrics`, `ClinicalAssessment` (colour card plus a static "Never make medical decisions based solely on AI" alert), `GradCAMPanel`, `InterpretationGuide`, `MedicalDisclaimer`, `DownloadButtons`, layout and UI primitives.
- `vite.config.js`: dev server on port 3000 with a `/api` proxy to `localhost:8000`, although `api.js` bypasses the proxy by using an absolute URL (hence the CORS wildcard).

`DownloadButtons` re-submits the image to `/report/*`, so each download triggers a fresh full inference plus Grad-CAM.

6. Data, Model and Configuration Details

Dataset. Not present and not described. The README says only "thyroid ultrasound or pathology images". The two sample files (`0000.jpg`, `0378.jpg`) have numeric names suggesting an indexed public dataset, but this is an inference and nothing in the repo confirms the source, size, class balance, splits or labelling method. The Streamlit help text claims a "minimum resolution 224x224" that the code does not enforce.

Classes. Single sigmoid output. 0 = Benign (Non-Cancerous), 1 = Malignant (Cancerous). Decision threshold fixed at 0.5.

Model (as inferable from code and README).

Property	Value	Source
Input	224 x 224 x 3, scaled to [0,1]	code
Output	1 unit, sigmoid	code/README
Framework	Keras 3 on TensorFlow, `.h5` file	code
Custom blocks	SEBlock, Avg2MaxPooling, DepthwiseSeparableConv	`model_loader.py`
Block detail	depthwise + pointwise conv, BN, swish, SE, residual	`model_loader.py`
Pooling	BN(avg - 2*max), pool 3, stride 2	`model_loader.py`
Regularisation	L2 = 0.001 in conv blocks	`model_loader.py`
Loss/optimiser	binary cross-entropy / Adam (compile-time only)	`model_loader.py`
Layers	"50 including custom sublayers"	README claim
Parameter count	not stated (README example shows 1,234,567 as illustration)	README

The architecture is a lightweight MobileNet/EfficientNet-style design with channel attention. Exact depth, filter counts and the classifier head are stored only in the remote `.h5` and are not visible here.

Thresholds.

Score p	Class	Risk text
p >= 0.75	Malignant	High Risk

Score p	Class	Risk text
0.50 <= p < 0.75	Malignant	Moderate Risk
0.25 < p < 0.50	Benign	Borderline
p <= 0.25	Benign	Low Risk

These cut-offs are hard-coded heuristics with no calibration or validation evidence in the repo.

Configuration and environment. No environment variables are read. Settings live in `config.py` and as literals: port 8000, host `0.0.0.0`, `reload=True`, CORS `allow_origins=["*"]` with `allow_credentials=True`, log file `logs/app.log`, model cache `./model_cache`, frontend dev port 3000. Network access to HuggingFace is required on first start (the model is cached afterwards, but `hf_hub_download` is still called every start). No version pin or revision hash is used for the model download.

7. Setup, Run and Deployment Instructions

Backend

1. Create and activate a virtual environment (`python -m venv venv`, then `venv\Scripts\activate` on Windows or `source venv/bin/activate`).
2. `pip install -r requirements.txt`. Add `requests` if you plan to run `test_api.py`.
3. Start the server: `python app.py` or `uvicorn app:app --host 0.0.0.0 --port 8000 --reload`. The first start downloads the model and runs a warm-up prediction.
4. Open `http://localhost:8000/docs` (Swagger) or `/redoc`.
5. Smoke test: `curl http://localhost:8000/api/v1/health`, or place `test_image.jpg` in the working directory and run `python test_api.py`.

Example request:

```
curl -X POST http://localhost:8000/api/v1/analyze \
  -H "Content-Type: application/json" \
  -d '{"image": "<base64>", "filename": "scan.jpg", "include_gradcam": true}'
```

Frontend

1. `cd frontend`, `npm install`.
2. `npm run dev` (Vite on port 3000). The backend must be on port 8000.
3. `npm run build` produces a static bundle; `npm run lint` runs ESLint with zero allowed warnings.

Streamlit (legacy)

1. `pip install -r requirements.txt`.
2. `streamlit run main_streamlit.py`. For Streamlit Cloud, `packages.txt` supplies the OpenGL/GLib libraries OpenCV needs.

Deployment. The repo contains no Dockerfile, CI configuration, process manager config, or reverse-proxy config. The README says FastAPI can run on "any server/cloud" and Streamlit on Streamlit Cloud. A production deployment would need to address: the hard-coded `localhost:8000` in `frontend/src/services/api.js`, `reload=True`, the wildcard CORS policy, TLS, and a pre-baked model cache to avoid network downloads at start-up.

8. Security, Privacy and Clinical-Safety Observations

Security and privacy:

- **No authentication or authorisation** on any endpoint, and **CORS allows every origin** (combined with `allow_credentials=True`). Anyone who can reach the port can submit images and obtain reports.

- **No rate limiting.** Each request runs TensorFlow inference (and Grad-CAM, and for reports a second full analysis), making the service easy to exhaust.
- **Input checks are weak.** Only the base64 string length is checked on the server; file type and magic bytes are not verified. PIL decodes arbitrary image formats, and there is no explicit decompression-bomb guard beyond Pillow's default warning threshold. The `ALLOWED_EXTENSIONS` constant is unused server-side.
- **Error disclosure.** The global exception handler returns `str(exc)` to clients, which can leak internal details; most route handlers do return generic messages.
- **PHI exposure.** Medical images and filenames (which may contain patient identifiers) are logged by name to `logs/app.log` and echoed into reports. Images are not persisted, but logs are. There is no encryption in transit configured, no audit trail with user identity, and no consent or retention policy. These facts matter for HIPAA/GDPR contexts.
- **Supply chain.** The model is fetched from a third-party HuggingFace repo with no revision pin or checksum, and loaded from `.h5` via Keras; loading untrusted HDF5/Keras models can be an arbitrary-code risk when custom or Lambda layers are involved. Dependencies are unpinned.
- **Client-supplied** `layer_name` is passed to `model.get_layer` unvalidated; a bad name yields an error, not code execution.

Clinical safety:

- Disclaimers are present and consistent: the frontend alert and `MedicalDisclaimer` component, the PDF disclaimer ("NOT FDA-approved"), the Streamlit footer, and the README.
- Despite that, the output language is directive ("Immediate specialist consultation and biopsy recommended", "Follow-up imaging in 6-12 months advised"). These recommendations are generated from uncalibrated score cut-offs with no clinical validation shown, and they could be read as medical advice.
- "Confidence percentage" is simply the sigmoid probability of the predicted class; it is not a calibrated confidence and is not accompanied by uncertainty estimates.
- There is no out-of-distribution or image-type check. A non-thyroid photo, a different ultrasound modality, or a blank image would still receive a Benign/Malignant label and a risk statement.
- Grad-CAM is always computed against the malignant output (see 5.6) and from a layer picked by name matching, so the displayed "attention" may be unfaithful or misleading, particularly for benign predictions. The `InterpretationGuide` presents red/yellow as "important regions", which overstates what Grad-CAM proves.
- The Streamlit app can display "LOW RISK" for borderline benign scores (bug in 5.12), contradicting the API.

9. Known Limitations and Suggested Improvements

Observed limitations and risks:

1. **Reproducibility gap.** No training code, notebooks, dataset reference, split definition, or evaluation outputs. Claims about performance cannot be verified from this repo.
2. **Metrics absent.** No accuracy, sensitivity, specificity, AUC, confusion matrix or calibration data is reported.
3. **Possible Grad-CAM layer failure.** Layer selection filters names containing `conv2d`. In the loaded model the convolutions live inside `DepthwiseSeparableConv` custom layers whose names are like `depthwise_separable_conv_N`, and the README's own `model-info` example lists those names under `conv_layers`, which contradicts the filter. If no top-level layer matches, `/gradcam` returns 400 and `/analyze` silently omits Grad-CAM. Whether this happens depends on the remote model and could not be confirmed here. Even if a match is found, it may be an early or intermediate layer rather than the last feature map.
4. **Dead and duplicated code.** `routes.py.old`, `model/custom_layers.py` (conflicting layer definitions, an unused LSTM), unused `ErrorResponse`, unused config constants, unused `pred_index`, and a `utils/__init__.py` that imports TensorFlow eagerly.

5 **Documentation drift.** README structure and endpoint list omit `routes/` package and `/report/*`; mentions "Custom LSTM Layers"; parameter count and conv layer names in examples look illustrative; migration notes reference an author-specific Windows path (`e:\thyroid with gradcam\...`).

6 **Inconsistent business logic.** Risk text is implemented three times (backend, Streamlit dict, frontend colour helper) with one buggy copy.

7 **Performance.** The model is loaded in a global with blocking `model.predict` calls inside `async def` handlers, which blocks the event loop; report endpoints repeat full inference; an extra forward pass is used for Grad-CAM; the heatmap is built with a per-channel Python loop.

8 **Operational.** Single worker, `reload=True` in `__main__`, deprecated `@app.on_event` handlers, hard-coded API URL in the frontend, unpinned dependencies, missing `requests` in requirements, no Docker or CI, no automated tests (the smoke script skips image tests by default).

9 **Preprocessing mismatch risk.** Resize uses LANCZOS on the whole image with aspect ratio distortion; `/255` scaling is assumed to match training but is not documented.

Suggested improvements:

- Publish the training notebook or scripts, dataset provenance and licence, split strategy, and full held-out metrics (sensitivity, specificity, ROC-AUC, PR curves, calibration) in a model card, and pin the HuggingFace revision with a checksum.
- Fix Grad-CAM: select the target layer deliberately (for example the last `DepthwiseSeparableConv` output), validate `layer_name` against the model, and for benign predictions show the complement or use `1-p`; add a faithfulness sanity check.
- Replace the heuristic risk bands with validated, calibrated thresholds, soften directive wording, and show uncertainty; add input-quality and OOD gating.
- Harden the API: authentication, restricted CORS origins, rate limiting, strict file-type and size validation on decoded bytes, run inference in a threadpool or worker, sanitise error messages, and avoid logging filenames or PHI.
- Remove dead files, unify risk logic in one shared module, fix the Streamlit threshold bug, move to FastAPI lifespan handlers, and make the frontend base URL configurable via `VITE_API_URL`.
- Add pytest unit tests (preprocessing, classify, assess_risk, schemas), an API integration test with a mock model, a Dockerfile, pinned requirements and CI.